	Marwadi University **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM:** To perform and analyze various fundamental DSP operations on a given, including color-to-grayscale conversion, image negative, logarithmic and power-law transformations, thresholding, adaptive thresholding, and RGB channel separation, and to visualize the results	
Experiment No: 07	**Date: 14/08/2026**	**Enrollment No:** 92400133147

1. Importing Required Python Libraries

Objectives:

To import the required Python libraries such as OpenCV, NumPy, and Matplotlib for performing image processing, numerical operations, and image visualization.

Applications:

These libraries are commonly used in digital image processing, computer vision, image analysis, and graphical visualization.

Code:

```
import cv2
import numpy as np
import matplotlib.pyplot as plt
print("Libraries imported successfully")
✓ 5.6s
```

Output:

```
[1]  ✓ 5.6s
...  Libraries imported successfully
```

Conclusion:
The required Python libraries, including OpenCV, NumPy, and Matplotlib, were successfully imported for image processing, numerical operations, and image visualization.

2. Reading and Loading the Input Image

Objectives:

To read and load the input image using OpenCV and determine its basic dimensions.

Applications:

Image loading is the first step in image-processing applications such as object detection, image enhancement, segmentation, and computer vision.

Code:

```python
image = cv2.imread(r"C:\Users\devah\Documents\ICT MU\Sem 5\DSIP\Lab\Images\PBKS vs MI.jpg")
if image is None:
    print("Error: Image not found")
else:
    print("Image loaded successfully")
    print("Image shape:", image.shape)
```

[2] ✓ 0.0s

Output:

```
... Image loaded successfully
    Image shape: (429, 715, 3)
```

Conclusion:
The input image was successfully loaded using OpenCV. The image dimensions and other basic properties were obtained for further processing.

3. RGB and Grayscale Image Conversion

Objectives:

To convert the input image from BGR format to RGB format for proper visualization and to convert the color image into a grayscale image for intensity-based processing.

Applications:

Grayscale conversion is widely used in image enhancement, thresholding, edge detection, segmentation, and feature extraction.

Code:

```python
image2 = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray_image = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
print("RGB and grayscale images created successfully")
```

[3] ✓ 0.0s

	Marwadi University **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM:** To perform and analyze various fundamental DSP operations on a given, including color-to-grayscale conversion, image negative, logarithmic and power-law transformations, thresholding, adaptive thresholding, and RGB channel separation, and to visualize the results	
Experiment No: 07	**Date: 14/08/2026**	**Enrollment No:** 92400133147

Output:

```
...   RGB and grayscale images created successfully
```

Conclusion:

The original BGR image was converted into RGB format for proper visualization using Matplotlib and into grayscale format for intensity-based image processing.

4. Display of Original Image

Objectives:

To display and save the original input image as a reference for comparing the results of different image-processing techniques.

Applications:

Original images are used as reference data in image enhancement, restoration, segmentation, and image-quality analysis.

Code:

```
cv2.imwrite(r"C:\Users\devah\Documents\ICT MU\Sem 5\DSIP\Lab\Lab 4 images\01_original_image.png", image)
plt.figure(figsize=(6,6))
plt.imshow(image2)
plt.title("Input Image")
plt.axis("off")
plt.show()
```
[4] ✓ 0.3s

Output:

Conclusion:

The original color image was successfully displayed and saved. This image serves as the reference image for the subsequent image-processing operations.

5. Conversion of Color Image to Grayscale

Objectives:

To convert and visualize the color image as a grayscale image containing intensity information.

Applications:

Grayscale images are used in thresholding, edge detection, object recognition, texture analysis, and other computationally efficient image-processing operations.

Code:

```
cv2.imwrite(r"C:\Users\devah\Documents\ICT MU\Sem 5\DSIP\Lab\Lab 4 images\02_grayscale_image.png",gray_image)
plt.figure(figsize=(6,6))
plt.imshow(gray_image, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")
plt.show()
```

Output:

Conclusion:

The color image was successfully converted into a grayscale image. The grayscale representation contains intensity information without separate color components and is suitable for further image-processing operations.

6. Image Negative Transformation

Objectives:

To generate the negative of the grayscale image by reversing its intensity values.

$$S = 255 - r$$

Applications:

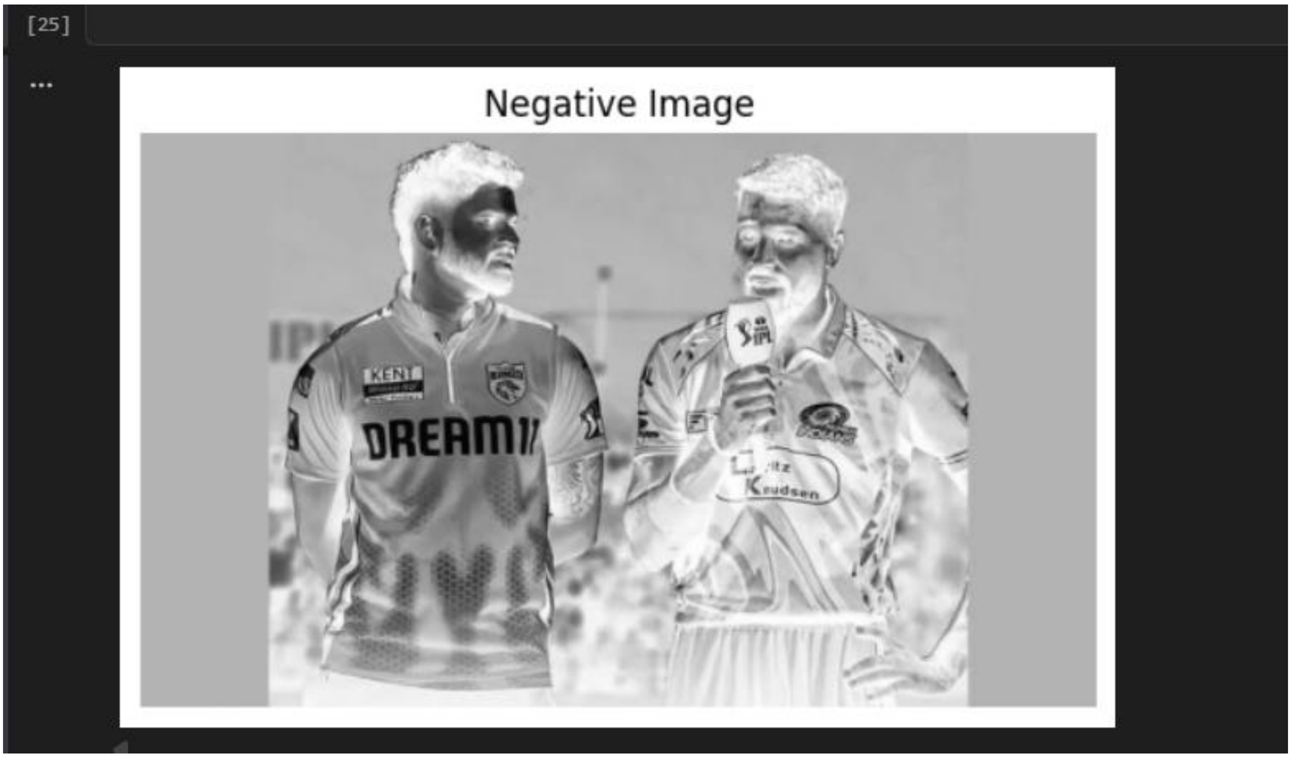
Image negatives are useful for enhancing details in certain images, particularly medical images, X-ray images, photographic negatives, and images containing dark features on bright backgrounds.

Code:

```
negative_image = 255 - gray_image
cv2.imwrite(r"C:\Users\devah\Documents\ICT MU\Sem 5\DSIP\Lab\Lab 4 images\03_negative_image.png",negative_image)
plt.figure(figsize=(6,6))
plt.imshow(negative_image, cmap="gray")
plt.title("Negative Image")
plt.axis("off")
plt.show()
```
[25]

	Marwadi University **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM:** To perform and analyze various fundamental DSP operations on a given, including color-to-grayscale conversion, image negative, logarithmic and power-law transformations, thresholding, adaptive thresholding, and RGB channel separation, and to visualize the results	
Experiment No: 07	**Date: 14/08/2026**	**Enrollment No:** 92400133147

Output:



Conclusion:

The negative transformation was successfully applied to the grayscale image. Dark regions were converted into bright regions and bright regions into dark regions, producing the negative representation of the original image.

7. Logarithmic Intensity Transformation

Objectives:

To apply logarithmic intensity transformation to enhance low-intensity details while compressing high-intensity values.

Applications:

Log transformation is useful for images with a large dynamic range, such as Fourier-spectrum images, astronomical images, and images where details in dark regions need enhancement.

Code:

```
max_pixel = int(np.max(gray_image))
c = 255 / np.log(1 + max_pixel)
log_image = c * np.log(1 + gray_image.astype(np.float32))
log_image = np.clip(log_image, 0, 255)
log_image = log_image.astype(np.uint8)
print("c =", c)
cv2.imwrite(r"C:\Users\devah\Documents\ICT MU\Sem 5\DSIP\Lab\Lab 4 images\04_log_image.png",log_image)
plt.figure(figsize=(6,6))
plt.imshow(log_image, cmap="gray")
plt.title("Log Transformation")
plt.axis("off")
plt.show()
```
6]

Output:

Conclusion:

The logarithmic transformation was successfully applied to the grayscale image. The transformation enhances lower-intensity pixel values and compresses higher-intensity values, improving the visibility of details in darker regions.

8. Power Law (Gamma) Transformation

Objectives:

To apply power-law (gamma) transformation to modify the brightness and contrast of the grayscale image.

Applications:

Gamma correction is widely used in display systems, photography, medical imaging, computer graphics, and image enhancement.

	Marwadi University **Faculty of Engineering &Technology** **Department of Information and Communication Technology**
Subject: Digital Signal and Image Processing (01CT1513)	**AIM:** To perform and analyze various fundamental DSP operations on a given, including color-to-grayscale conversion, image negative, logarithmic and power-law transformations, thresholding, adaptive thresholding, and RGB channel separation, and to visualize the results

Experiment No: 07	**Date: 14/08/2026**	**Enrollment No:** 92400133147

Code:

```
gamma = 0.5
gamma_image = 255 * (gray_image.astype(np.float32) / 255) ** gamma
gamma_image = np.clip(gamma_image, 0, 255)
gamma_image = gamma_image.astype(np.uint8)
cv2.imwrite(r"C:\Users\devah\Documents\ICT MU\Sem 5\DSIP\Lab\Lab 4 images\05_gamma_image.png",gamma_image)
plt.figure(figsize=(6,6))
plt.imshow(gamma_image, cmap="gray")
plt.title("Power Law Transformation")
plt.axis("off")
plt.show()
```
[27]

Output:

[27]

Conclusion:

The power-law transformation was successfully implemented with . The transformation modifies image brightness and contrast by applying a nonlinear relationship between the input and output intensity values.

9. Binary Thresholding with Threshold Value 147

Objectives:

To convert the grayscale image into a binary image using a threshold value of 147.

Applications:

Binary thresholding is used for image segmentation, object-background separation, document processing, character recognition, and shape detection.

Code:

```
threshold_value = 147
_, thresh_image = cv2.threshold(gray_image,threshold_value,255,cv2.THRESH_BINARY)
cv2.imwrite(r"C:\Users\devah\Documents\ICT MU\Sem 5\DSIP\Lab\Lab 4 images\06_threshold_147.png",thresh_image)
plt.figure(figsize=(6,6))
plt.imshow(thresh_image, cmap="gray")
plt.title("Threshold Image - Threshold = 147")
plt.axis("off")
plt.show()
```

Output:

Conclusion:

Binary thresholding was successfully performed using a threshold value of 147. Pixels above the threshold were converted to white, while pixels below the threshold were converted to black, producing a binary image.

10. Binary Thresholding with Threshold Value 80

Objectives:

To study the effect of changing the threshold value on binary image formation by applying a threshold of 80.

Applications:

Different threshold values can be selected to separate objects from backgrounds under different lighting and intensity conditions.

Code:

```
threshold_value = 80
_, thresh_image_80 = cv2.threshold(gray_image,threshold_value,255,cv2.THRESH_BINARY)
cv2.imwrite(r"C:\Users\devah\Documents\ICT MU\Sem 5\DSIP\Lab\Lab 4 images\07_threshold_80.png",thresh_image_80)
plt.figure(figsize=(6,6))
plt.imshow(thresh_image_80, cmap="gray")
plt.title("Threshold Image - Threshold = 80")
plt.axis("off")
plt.show()
```
[29]

<table>
<tr><td></td><td colspan="2">Marwadi University
Faculty of Engineering &Technology
Department of Information and Communication Technology</td></tr>
<tr><td>Subject: Digital Signal and Image Processing (01CT1513)</td><td colspan="2">AIM: To perform and analyze various fundamental DSP operations on a given, including color-to-grayscale conversion, image negative, logarithmic and power-law transformations, thresholding, adaptive thresholding, and RGB channel separation, and to visualize the results</td></tr>
<tr><td>Experiment No: 07</td><td>Date: 14/08/2026</td><td>Enrollment No: 92400133147</td></tr>
</table>

Output:

Conclusion:

Binary thresholding was performed again using a threshold value of 80. Changing the threshold value altered the regions classified as foreground and background, demonstrating the effect of threshold selection on image segmentation.

11. Adaptive Mean Thresholding

Objectives:

To perform adaptive thresholding using the local mean intensity of neighboring pixels instead of using one fixed global threshold.

Applications:

Adaptive thresholding is useful for document scanning, OCR, text extraction, and images with non-uniform illumination.

	Marwadi University **Faculty of Engineering &Technology** **Department of Information and Communication Technology**	
Subject: Digital Signal and Image Processing (01CT1513)	**AIM:** To perform and analyze various fundamental DSP operations on a given, including color-to-grayscale conversion, image negative, logarithmic and power-law transformations, thresholding, adaptive thresholding, and RGB channel separation, and to visualize the results	
Experiment No: 07	**Date: 14/08/2026**	**Enrollment No:** 92400133147

Code:

```python
thresh_image = cv2.adaptiveThreshold(gray_image,255,cv2.ADAPTIVE_THRESH_MEAN_C,cv2.THRESH_BINARY,11,2)
cv2.imwrite(r"C:\Users\devah\Documents\ICT MU\Sem 5\DSIP\Lab\Lab 4 images\08_adaptive_mean_threshold.png",thresh_image)
plt.figure(figsize=(6,6))
plt.imshow(thresh_image, cmap="gray")
plt.title("Adaptive Mean Thresholding")
plt.axis("off")
plt.show()
```

[30] Python

Output:

Conclusion:

Adaptive mean thresholding was successfully applied to the grayscale image. Unlike global thresholding, the threshold is calculated locally using neighboring pixels, making it more suitable for images with non-uniform illumination.

12. Analysis of Image Properties

Objectives:

To determine the basic properties of the input image, including its shape, height, width, number of channels, and data type.

Applications:

Image-property analysis is important when selecting appropriate image-processing algorithms, memory requirements, image dimensions, and color-processing methods.

Code:

```
print("Shape:", image.shape)
print("Height:", image.shape[0])
print("Width:", image.shape[1])
print("Channels:", image.shape[2])
print("Data type:", image.dtype)
```

[34]

Output:

[34]

```
...  Shape: (429, 715, 3)
     Height: 429
     Width: 715
     Channels: 3
     Data type: uint8
```

Conclusion:

The basic properties of the input image, including its shape, height, width, number of color channels, and data type, were successfully determined. These properties provide important information about the structure and representation of the image.

13. Loading the RGB Color Image

Objectives:

To load the RGB color demonstration image that will be used for separating and analyzing the individual color channels.

Applications:

RGB images are fundamental in digital photography, computer graphics, display systems, image processing, and computer vision.

Code:

```
image1 = cv2.imread(r"C:\Users\devah\Downloads\ChatGPT Image Aug 13, 2026, 10_34_53 AM.png")
if image is None:
    print("Error: Image not found")
else:
    print("Image loaded successfully")
    print("Image shape:", image.shape)
[47]
```

Output:

```
[47]
...  Image loaded successfully
     Image shape: (429, 715, 3)
```

Conclusion:

The RGB demonstration image was successfully loaded using OpenCV. This image was used to demonstrate the individual red, green, and blue color components.

14. Separation and Visualization of RGB Channels

Objectives:

To separate the RGB image into its individual red, green, and blue channels and visualize the contribution of each channel.

Applications:

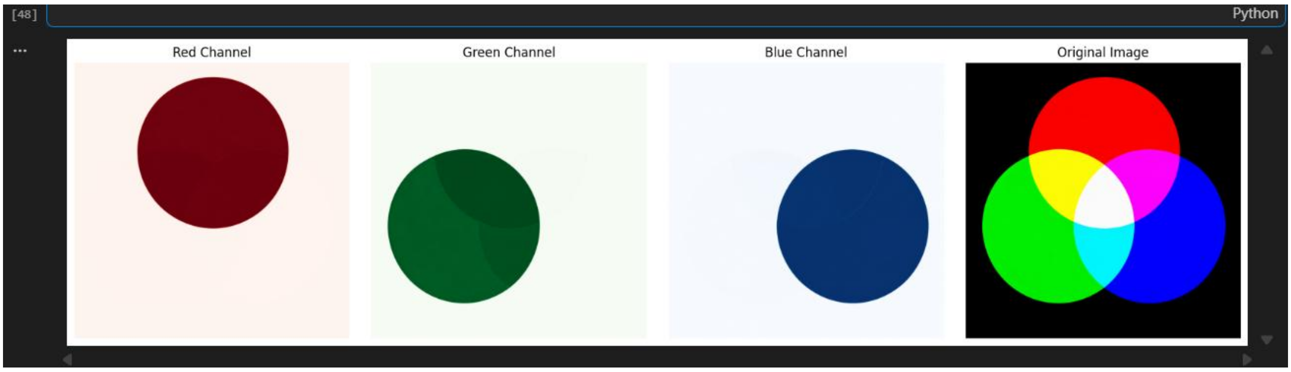
RGB channel separation is used in color image analysis, color-based segmentation, object detection, image enhancement, color correction, and computer vision.

Code:

```
b, g, r = cv2.split(image1)
original = cv2.cvtColor(image1, cv2.COLOR_BGR2RGB)

plt.figure(figsize=(16, 4))

plt.subplot(1, 4, 1)
plt.imshow(r, cmap='Reds')
plt.axis('off')
plt.title('Red Channel')

plt.subplot(1, 4, 2)
plt.imshow(g, cmap='Greens')
plt.axis('off')
plt.title('Green Channel')

plt.subplot(1, 4, 3)
plt.imshow(b, cmap='Blues')
plt.axis('off')
plt.title('Blue Channel')

plt.subplot(1, 4, 4)
plt.imshow(original)
plt.axis('off')
plt.title('Original Image')

red_image = np.zeros_like(image1)
red_image[:, :, 2] = r
green_image = np.zeros_like(image1)
green_image[:, :, 1] = g
blue_image = np.zeros_like(image1)
blue_image[:, :, 0] = b

cv2.imwrite(r"C:\Users\devah\Documents\ICT MU\Sem 5\DSIP\Lab\Lab 4 images\09_red_image.png", red_image)
cv2.imwrite(r"C:\Users\devah\Documents\ICT MU\Sem 5\DSIP\Lab\Lab 4 images\10_green_image.png", green_image)
cv2.imwrite(r"C:\Users\devah\Documents\ICT MU\Sem 5\DSIP\Lab\Lab 4 images\11_blue_image.png", blue_image)

plt.tight_layout()
plt.show()
```

[48]

Output:



Conclusion:

The image was successfully separated into red, green, and blue channels. Each channel represents the intensity contribution of its respective primary color. The individual channels and the original color image were visualized together, demonstrating how RGB channels combine to form a color image.

Conclusion:

The experiment successfully demonstrated several fundamental Digital Image Processing techniques using Python and OpenCV. The input image was loaded, analyzed, converted into different representations, and processed using various intensity transformation and thresholding techniques.
The experiment included RGB and grayscale conversion, negative transformation, logarithmic transformation, power-law transformation, binary thresholding, adaptive thresholding, image-property analysis, and RGB channel separation.
It was observed that different image-processing techniques produce different visual effects and are useful for different applications. Intensity transformations modify the brightness and contrast of an image, while thresholding techniques help separate image regions based on pixel intensity. RGB channel separation demonstrated that a color image is composed of three primary color components—red, green, and blue.
Overall, the experiment provided practical understanding of fundamental image-processing operations and their implementation using OpenCV, NumPy, and Matplotlib in Python.

GitHub:

https://github.com/Devaharshagoud/DSIP-Experiments.git